DATABASE MANAGEMENT SYSTEMS | 23A05402T

UNIT - III : SQL (Structured Query Language)

Easy handwritten notes with diagrams for students

Topics in this unit

1. Basic SQL querying: SELECT, project, WHERE, operators
2. SQL functions: date and time, numeric, string, conversion
3. Creating tables with relationships; key and integrity constraints
4. Grouping, aggregation, HAVING and ordering
5. Joins (inner, outer, cross, self)
6. Nested queries and sub queries
7. Views (updatable and non-updatable)
8. Relational set operations

* **Study tip:** Every topic has a small diagram or table. Learn the **RED words** first, then draw the diagram once from memory - that is the fastest way to remember it for exams.

Syllabus (as given)

SQL: Basic SQL querying (select and project) using where clause, arithmetic & logical operations, SQL functions (Date and Time, Numeric, String conversion). Creating tables with relationship, implementation of key and integrity constraints, nested queries, sub queries, grouping, aggregation, ordering, implementation of different types of joins, view (updatable and non-updatable), relational set operations.

Part I : Basic SQL Querying

All examples use these tables (from Unit II):

```
DEPT    (Dept_id, Dname, Location)
STUDENT (Roll_no, Name, Age, Email, Dept_id, Fees)
COURSE  (Course_id, Title, Credits)
ENROLLS (Roll_no, Course_id, Grade)
```

1.1 SELECT statement (Select = Project)

SELECT retrieves data. The column list is the relational-algebra **projection** (π) and the **WHERE** clause is the **selection** (σ).

```
SELECT column_list | *          -- what columns (projection)
FROM   table_list               -- from which tables
WHERE  condition                -- which rows (selection)
GROUP BY columns                -- make groups
HAVING group_condition          -- which groups
ORDER BY columns [ASC|DESC]     -- sort result
LIMIT n;                        -- first n rows (MySQL/PostgreSQL)
```

```
SELECT * FROM STUDENT;                          -- all columns, all rows
SELECT Name, Age FROM STUDENT;                  -- project two columns
SELECT DISTINCT Dept_id FROM STUDENT;           -- remove duplicates
SELECT Name AS Student_Name, Fees/12 AS Monthly FROM STUDENT;   -- alias + arithmetic
SELECT Name FROM STUDENT WHERE Age > 20;        -- select rows
```

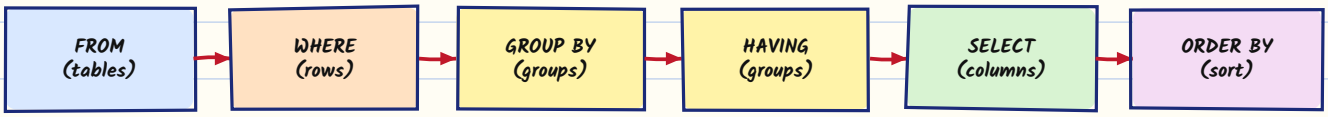


Fig 3.1 : Logical order in which SQL executes a query (not the order it is written)

1.2 WHERE clause - Arithmetic, Comparison and Logical operators

Type	Operators	Example
Arithmetic	+ - * / % (MOD)	SELECT Name, Fees*1.1 FROM STUDENT;
Comparison	= <> (or !=) < > <= >=	WHERE Age >= 20
Logical	AND OR NOT	WHERE Age > 18 AND Dept_id = 'D1'
Range	BETWEEN a AND b (inclusive)	WHERE Age BETWEEN 18 AND 22
List	IN (v1, v2...) / NOT IN	WHERE Dept_id IN ('D1','D2')
Pattern	LIKE with % (any string), _ (one char)	WHERE Name LIKE 'R%' / '_a%'
Null test	IS NULL / IS NOT NULL	WHERE Email IS NULL

* **Precedence:** ( ) first, then * / %, then + -, then comparison, then **NOT**, then **AND**, then **OR**. Use brackets to be safe.

```
SELECT Name FROM STUDENT WHERE Name LIKE 'A%';      -- starts with A
SELECT Name FROM STUDENT WHERE Name LIKE '%n';      -- ends with n
SELECT Name FROM STUDENT WHERE Name LIKE '_a%';     -- second letter a
```

```
SELECT * FROM STUDENT WHERE (Age < 19 OR Age > 21) AND Dept_id = 'D1';
```

1.3 ORDER BY and LIMIT

```
SELECT Name, Fees FROM STUDENT ORDER BY Fees DESC;          -- highest first
SELECT * FROM STUDENT ORDER BY Dept_id ASC, Age DESC;       -- multi-column
SELECT * FROM STUDENT ORDER BY Fees DESC LIMIT 3;           -- top 3 (Oracle: FETCH FIRST 3 ROWS ONLY)
```

ASC (ascending) is the default. NULLs sort first or last depending on the DBMS.

Part 2 : SQL Functions

Functions take values and return a value. They are of two kinds: **single-row functions** (one result per row - below) and **aggregate functions** (one result per group - Part 4). Names differ slightly between MySQL, Oracle and PostgreSQL.

2.1 Numeric functions

Function	Purpose	Example -> Result
ABS(x)	Absolute value	ABS(-7) -> 7
ROUND(x, d)	Round to d decimals	ROUND(15.678, 1) -> 15.7
CEIL(x) / FLOOR(x)	Round up / down	CEIL(4.2) -> 5 ; FLOOR(4.8) -> 4
MOD(x, y)	Remainder	MOD(10, 3) -> 1
POWER(x, y)	x raised to y	POWER(2, 5) -> 32
SQRT(x)	Square root	SQRT(49) -> 7
TRUNC(x, d) / TRUNCATE	Cut decimals (no rounding)	TRUNC(15.79, 1) -> 15.7
SIGN(x)	-1, 0 or 1	SIGN(-5) -> -1

2.2 String functions

Function	Purpose	Example -> Result
UPPER(s) / LOWER(s)	Change case	UPPER('ravi') -> RAVI
LENGTH(s)	Number of characters	LENGTH('Sita') -> 4
CONCAT(a, b) or a || b	Join strings	CONCAT('Data','base') -> Database
SUBSTR(s, start, len)	Part of a string	SUBSTR('DATABASE', 1, 4) -> DATA
INSTR(s, sub)	Position of substring	INSTR('DATABASE','BASE') -> 5
TRIM(s) / LTRIM / RTRIM	Remove spaces	TRIM(' hi ') -> hi
REPLACE(s, old, new)	Replace text	REPLACE('cat','c','b') -> bat
LPAD(s, n, ch) / RPAD	Pad to length n	LPAD('7', 3, '0') -> 007
REVERSE(s)	Reverse string	REVERSE('abc') -> cba

2.3 Date and Time functions

Function	Purpose	Example
CURRENT_DATE / CURDATE()	Today's date	SELECT CURRENT_DATE;

Function	Purpose	Example
NOW() / SYSDATE / CURRENT_TIMESTAMP	Current date + time	SELECT NOW();
EXTRACT(part FROM d)	Year / month / day of a date	EXTRACT(YEAR FROM DOB)
YEAR(d), MONTH(d), DAY(d)	Parts (MySQL)	YEAR('2026-10-15') -> 2026
DATE_ADD(d, INTERVAL n unit)	Add time to date	DATE_ADD(d, INTERVAL 7 DAY)
ADD_MONTHS(d, n)	Add months (Oracle)	ADD_MONTHS(SYSDATE, 3)
DATEDIFF(d1, d2)	Days between dates	DATEDIFF('2026-10-15','2026-10-01') -> 14
LAST_DAY(d)	Last date of the month	LAST_DAY('2026-02-10') -> 2026-02-28
DATE_FORMAT(d, fmt) / TO_CHAR(d, fmt)	Display in a format	DATE_FORMAT(d,'%d-%m-%Y')

2.4 Conversion functions

Function	Purpose	Example
CAST(x AS type)	Standard type conversion	CAST('123' AS INT) -> 123
CONVERT(x, type)	MySQL / SQL Server conversion	CONVERT('2026-10-15', DATE)
TO_CHAR(x, fmt)	Number/date -> text (Oracle, PostgreSQL)	TO_CHAR(SYSDATE,'DD-MON-YYYY')
TO_NUMBER(s)	Text -> number	TO_NUMBER('45.5') -> 45.5
TO_DATE(s, fmt)	Text -> date	TO_DATE('15-10-2026','DD-MM-YYYY')
COALESCE(a, b, ...) / NVL(a, b)	First non-NULL value	COALESCE(Email, 'none')

```
SELECT UPPER(Name), ROUND(Fees/3, 2), LENGTH(Name) FROM STUDENT;
SELECT Name, COALESCE(Email, 'not given') FROM STUDENT;      -- replace NULL
SELECT Name, CAST(Fees AS INT) FROM STUDENT;
```

Part 3 : Creating Tables with Relationships; Constraints

Relationships from the ER diagram are implemented using **primary keys** and **foreign keys**.

Relationship	How to implement	Example
One-to-One (1:1)	FK in either table + UNIQUE (or share the same PK)	PERSON - PASSPORT: Passport.Person_id UNIQUE FK
One-to-Many (1:N)	Put FK on the 'many' side referring to the 'one' side	DEPT (1) - STUDENT (N): STUDENT.Dept_id FK
Many-to-Many (M:N)	New **junction (bridge) table** with FKs of both; PK = both FKs	STUDENT - COURSE -> ENROLLS(Roll_no, Course_id)

```
-- 1:N  (one department, many students)
CREATE TABLE STUDENT (
    Roll_no   INT PRIMARY KEY,
    Name      VARCHAR(40) NOT NULL,
    Dept_id   CHAR(2),
    CONSTRAINT fk_dept FOREIGN KEY (Dept_id) REFERENCES DEPT(Dept_id)
);
```

```
-- M:N  (junction table)
CREATE TABLE ENROLLS (
    Roll_no    INT,
    Course_id  VARCHAR(6),
    Grade      CHAR(1) CHECK (Grade IN ('A','B','C','D','F')),
    PRIMARY KEY (Roll_no, Course_id),                    -- composite key
    FOREIGN KEY (Roll_no)   REFERENCES STUDENT(Roll_no) ON DELETE CASCADE,
    FOREIGN KEY (Course_id) REFERENCES COURSE(Course_id) ON DELETE CASCADE
);
```

3.1 Implementing key and integrity constraints

Constraint	Implemented by	Enforces
Entity integrity	PRIMARY KEY	Unique, not NULL row identity
Key / uniqueness	UNIQUE	No duplicate values in a column
Domain	data type, NOT NULL, CHECK, DEFAULT	Valid values only
Referential integrity	FOREIGN KEY ... REFERENCES	Child value must exist in parent
Referential actions	ON DELETE / ON UPDATE CASCADE, SET NULL, RESTRICT	What happens to child rows
General rules	CHECK (condition), ASSERTION / TRIGGER	Business rules

```
ALTER TABLE STUDENT ADD CONSTRAINT uq_email UNIQUE (Email);
ALTER TABLE STUDENT ADD CONSTRAINT ck_age  CHECK (Age BETWEEN IS AND 60);
ALTER TABLE STUDENT DROP CONSTRAINT uq_email;
-- Violations are rejected, e.g.:
INSERT INTO STUDENT (Roll_no, Name, Dept_id) VALUES (101, 'X', 'D9');   -- ERROR: D9 not in DEPT
```

Part 4 : Aggregation, Grouping and HAVING

4.1 Aggregate functions

Function	Returns	Notes
COUNT(*)	Number of rows	Counts NULLs too
COUNT(col)	Number of non-NULL values	COUNT(DISTINCT col) for unique values
SUM(col)	Total	Ignores NULL
AVG(col)	Average	Ignores NULL
MIN(col) / MAX(col)	Smallest / largest	Works on numbers, text, dates

```
SELECT COUNT(*) FROM STUDENT;                    -- total students
SELECT AVG(Fees), MAX(Age), MIN(Age) FROM STUDENT;
```

4.2 GROUP BY and HAVING

GROUP BY divides rows into groups with the same value; the aggregate is computed for each group. **HAVING** filters groups (after grouping); **WHERE** filters rows (before grouping).

```
SELECT Dept_id, COUNT(*) AS Total, AVG(Fees) AS Avg_fee
FROM   STUDENT
```

```
WHERE  Age >= 18                -- filter ROWS first
GROUP  BY Dept_id               -- one row per department
HAVING COUNT(*) > 2              -- filter GROUPS
ORDER  BY Total DESC;
```

WHERE	HAVING
Filters individual rows	Filters groups
Used before GROUP BY	Used after GROUP BY
Cannot use aggregate functions	Can use aggregate functions

* **Rule:** Every column in SELECT that is **not inside an aggregate** must appear in the **GROUP BY** clause.

Part 5 : Joins

A **join** combines rows of two or more tables using a related column (usually PK-FK).

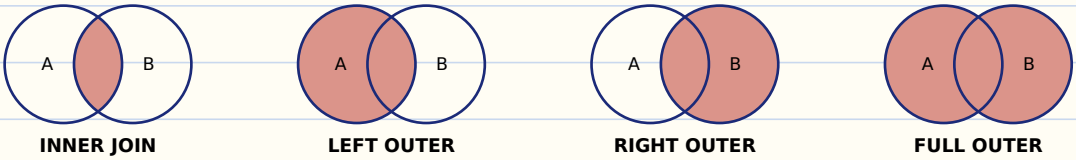


Fig 3.2 : Joins as Venn diagrams (shaded part = rows returned)

Join	Result	Syntax
CROSS JOIN	Cartesian product: every row with every row (m x n)	FROM A CROSS JOIN B
INNER JOIN	Only matching rows from both tables	FROM A INNER JOIN B ON A.x = B.x
NATURAL JOIN	Inner join on all columns with the same name	FROM A NATURAL JOIN B
LEFT OUTER JOIN	All rows of A + matching rows of B (NULL if none)	FROM A LEFT JOIN B ON ...
RIGHT OUTER JOIN	All rows of B + matching rows of A	FROM A RIGHT JOIN B ON ...
FULL OUTER JOIN	All rows of both; NULL where no match	FROM A FULL JOIN B ON ...
SELF JOIN	A table joined with itself using aliases	FROM EMP E1 JOIN EMP E2 ON E1.Mgr = E2.Id

```
-- INNER: student names with department names
SELECT S.Name, D.Dname
FROM   STUDENT S INNER JOIN DEPT D ON S.Dept_id = D.Dept_id;

-- LEFT OUTER: all departments, even those with no students
SELECT D.Dname, S.Name
FROM   DEPT D LEFT JOIN STUDENT S ON D.Dept_id = S.Dept_id;

-- Join three tables (M:N)
SELECT S.Name, C.Title, E.Grade
FROM   STUDENT S JOIN ENROLLS E ON S.Roll_no = E.Roll_no
               JOIN COURSE  C ON E.Course_id = C.Course_id;
```

```
-- SELF JOIN: employee with manager name
SELECT E.Name AS Employee, M.Name AS Manager
FROM   EMP E LEFT JOIN EMP M ON E.Mgr_id = M.Emp_id;
```

* **Remember:** Use **ON** for join conditions and **WHERE** for row filters. Forgetting the join condition gives a **cross product** with wrong, huge results.

Part 6 : Nested Queries and Sub Queries

A **subquery** is a SELECT inside another query (in parentheses). The inner query runs first and passes its result to the outer query.

Type	Description	Example
Scalar subquery	Returns ONE value; used with = < >	WHERE Fees > (SELECT AVG(Fees) FROM STUDENT)
Multi-row (IN)	Returns a list; used with IN / NOT IN / ANY / ALL	WHERE Dept_id IN (SELECT Dept_id FROM DEPT WHERE Location='Block A')
Correlated	Inner query uses a column of the outer query; runs once per outer row	WHERE EXISTS (SELECT 1 FROM ENROLLS E WHERE E.Roll_no = S.Roll_no)
FROM subquery	Result used as a temporary table (needs alias)	FROM (SELECT Dept_id, AVG(Fees) a FROM STUDENT GROUP BY Dept_id) T

```
-- students paying more than the average fee
SELECT Name FROM STUDENT WHERE Fees > (SELECT AVG(Fees) FROM STUDENT);

-- students of departments located in Block A
SELECT Name FROM STUDENT
WHERE Dept_id IN (SELECT Dept_id FROM DEPT WHERE Location = 'Block A');

-- students who are enrolled in at least one course   (EXISTS)
SELECT Name FROM STUDENT S
WHERE EXISTS (SELECT * FROM ENROLLS E WHERE E.Roll_no = S.Roll_no);

-- students NOT enrolled in any course
SELECT Name FROM STUDENT
WHERE Roll_no NOT IN (SELECT Roll_no FROM ENROLLS);

-- student(s) with the highest fee      (ALL)
SELECT Name FROM STUDENT WHERE Fees >= ALL (SELECT Fees FROM STUDENT);

-- 2nd highest fee (nested)
SELECT MAX(Fees) FROM STUDENT WHERE Fees < (SELECT MAX(Fees) FROM STUDENT);
```

Operator	Meaning
IN / NOT IN	Value is (not) in the subquery result list
ANY / SOME	Condition true for at least one value of the list
ALL	Condition true for every value of the list
EXISTS / NOT EXISTS	True if the subquery returns at least one row (any)

* **Beware:** **NOT IN** returns no rows if the subquery list contains a **NULL**. Prefer **NOT EXISTS** in such cases.

Part 7 : Relational Set Operations

Set operations combine the results of two SELECT queries. Both queries must have the **same number of columns** with **compatible data types**. They remove duplicates by default.

Operation	Result	Algebra
UNION	All rows of both queries, duplicates removed (UNION ALL keeps duplicates)	$R \cup S$
INTERSECT	Rows present in both queries	$R \cap S$
EXCEPT (MINUS in Oracle)	Rows of first query not in second	$R - S$

```
SELECT Roll_no FROM ENROLLS WHERE Course_id = 'C1'
UNION                                  -- enrolled in C1 OR C2
SELECT Roll_no FROM ENROLLS WHERE Course_id = 'C2';

SELECT Roll_no FROM ENROLLS WHERE Course_id = 'C1'
INTERSECT                               -- enrolled in C1 AND C2
SELECT Roll_no FROM ENROLLS WHERE Course_id = 'C2';

SELECT Roll_no FROM STUDENT
EXCEPT                                 -- students with NO enrolment
SELECT Roll_no FROM ENROLLS;
```

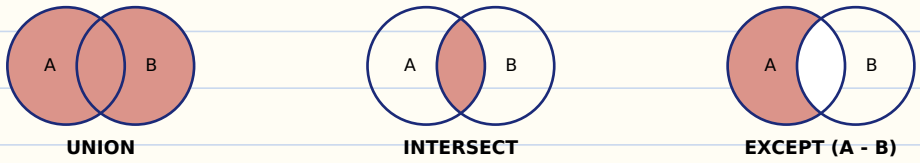


Fig 3.3 : Set operations on two query results A and B

Part 8 : Views

A **view** is a **virtual table** defined by a stored SELECT query. It holds no data itself; the query runs each time the view is used. Uses: **security** (hide columns / rows), **simplicity** (hide complex joins), **logical data independence**.

```
CREATE VIEW CSE_STUDENTS AS
    SELECT Roll_no, Name, Age FROM STUDENT WHERE Dept_id = 'D1';

SELECT * FROM CSE_STUDENTS WHERE Age > 19;          -- use like a table
CREATE OR REPLACE VIEW CSE_STUDENTS AS SELECT ... ;  -- change definition
DROP VIEW CSE_STUDENTS;                              -- remove view (data stays)
```

Updatable and non-updatable views

An **updatable view** allows INSERT / UPDATE / DELETE, which are passed to the base table. A **non-updatable (read-only) view** allows only SELECT.

Updatable view (simple)	Non-updatable view if it has ...
Based on ONE base table	Joins of several tables (usually)

Updatable view (simple)	Non-updatable view if it has ...
Includes the primary key	DISTINCT
No aggregate functions	Aggregate functions (SUM, AVG, COUNT ...)
No GROUP BY / HAVING	GROUP BY or HAVING
All NOT NULL base columns present	Set operations (UNION etc.) or subquery in FROM
Columns are real columns, not expressions	Computed / derived columns

```
CREATE VIEW ADULTS AS
    SELECT Roll_no, Name, Age FROM STUDENT WHERE Age >= 18
    WITH CHECK OPTION;       -- rejects updates that would make a row disappear from the view

UPDATE ADULTS SET Name = 'Ravi K' WHERE Roll_no = 101;    -- OK (updates STUDENT)
UPDATE ADULTS SET Age = 15 WHERE Roll_no = 101;           -- ERROR due to CHECK OPTION
```

* **Remember:** Views = **virtual tables**; **materialized views** store the result physically and must be refreshed.

Quick Revision - Unit III

- SELECT (projection) ... FROM ... WHERE (selection) ... GROUP BY ... HAVING ... ORDER BY.
- Operators: arithmetic, comparison, logical (AND/OR/NOT), BETWEEN, IN, LIKE (% and _), IS NULL.
- Functions: numeric (ROUND, CEIL, MOD), string (UPPER, SUBSTR, CONCAT), date (NOW, DATEDIFF, EXTRACT), conversion (CAST, TO_CHAR, TO_DATE).
- 1:N = FK on many side; M:N = junction table with composite PK; constraints PK, FK, UNIQUE, CHECK, NOT NULL, DEFAULT.
- Aggregates: COUNT, SUM, AVG, MIN, MAX. WHERE filters rows, HAVING filters groups.
- Joins: inner, left/right/full outer, cross, natural, self.
- Subqueries: scalar, IN, ANY/ALL, EXISTS, correlated, FROM-subquery.
- Set operations: UNION, INTERSECT, EXCEPT / MINUS (same columns needed).
- View = virtual table; updatable only if simple (one table, no aggregate / DISTINCT / GROUP BY).

Possible exam questions

1. Explain the SELECT statement with WHERE, GROUP BY, HAVING and ORDER BY with examples.
2. Explain the numeric, string, date/time and conversion functions in SQL.
3. How are 1:1, 1:N and M:N relationships implemented? Show SQL for tables with constraints.
4. Differentiate WHERE and HAVING. Explain aggregate functions.
5. Explain different types of joins with examples.
6. What are nested queries / correlated subqueries? Explain IN, EXISTS, ANY, ALL.
7. Explain UNION, INTERSECT and EXCEPT.
8. What is a view? Explain updatable and non-updatable views with examples.